Master Architecture Document

DRS High-Precision Distributed Synchronization Cluster

Consolidated Version 2.1 (Reviewed & Clarified)

May 14, 2026

---

1. Project Definition

1.1 Mission Statement

The DRS project establishes a deterministic, high-precision distributed global time base across a dynamic cluster of Raspberry Pi nodes operating exclusively in Linux user space.

The system shall maintain a physical synchronization delta of:

```
Δtphysical < 100 µs
```

as measured externally using GPIO-generated synchronization pulses observed on a logic analyzer or oscilloscope.

The architecture prioritizes:

- deterministic behavior
- operational simplicity
- fail-safe recovery
- real-time scheduling integrity
- minimal protocol complexity
- bounded timing behavior

1.2 Core Design Philosophy

The synchronization subsystem SHALL follow the KISS principle:

```
Keep It Simple, Stupid (KISS)
```

The protocol intentionally avoids:

- distributed consensus frameworks
- dynamic spanning trees
- kernel modifications

- external time authorities
- heavyweight serialization
- multi-hop routing
- dynamic routing protocols
- cryptographic processing in the synchronization hot path

The synchronization cluster is designed exclusively for:

- single-hop Layer-2 Gigabit Ethernet
- trusted laboratory environments
- fail-stop node behavior
- deterministic LAN conditions

1.3 Revised Core Constraints

- Wired Gigabit Ethernet SHALL be the exclusive synchronization transport.
- WLAN MAY be used exclusively for out-of-band management and provisioning.
- CPU Core 3 SHALL be reserved exclusively for synchronization operations.
- WLAN interrupts SHALL NEVER execute on Core 3.
- Ethernet IRQs SHALL NEVER execute on Core 3.
- The Linux host system clock SHALL NEVER be modified.
- All synchronization adjustments SHALL occur exclusively inside the virtual clock layer.

Explicitly prohibited:

- `clock_settime()`
- `adjtimex()`
- `CLOCK_REALTIME` stepping
- kernel PLL interaction

1.4 Synchronization Paradigm

The system implements:

```
Internal Monotonic Synchronization
```

A single elected leader defines the cluster reference timeline.

Follower nodes estimate:

- phase offset
- frequency drift
- path delay

relative to the elected leader.

The synchronization domain is entirely internal.

The cluster SHALL NOT synchronize to:

- UTC
- NTP
- GPS
- PTP grandmasters
- wall-clock time

1.5 Virtual Clock Model

The virtual clock mapping SHALL be defined as:

```
Tglobal = (Tlocal_raw × Rate) + Offset − LatencyCorrection_ns
```

Where:

Parameter	Description
Tlocal_raw	CLOCK_MONOTONIC_RAW timestamp
Rate	Fixed-point frequency scaling multiplier
Offset	Phase correction in nanoseconds
LatencyCorrection_ns	Static calibration compensation

Rate Representation

`Rate` SHALL be represented as signed 64-bit Q32.32 fixed-point.

Definitions:

Value	Meaning
0x00000001_00000000	1.0x nominal clock rate
+1000 ppm	Maximum positive slew
-1000 ppm	Maximum negative slew

Floating-point arithmetic SHALL NOT be used inside the synchronization hot path.

1.6 Time Base Definition

The synchronization engine SHALL operate exclusively on:

```
CLOCK_MONOTONIC_RAW
```

All protocol timestamps SHALL represent:

```
Absolute nanoseconds since local kernel boot.
```

The cluster timeline SHALL be defined relative to the elected leader's monotonic boot-time origin.

The system SHALL NOT use:

- Unix Epoch
- wall-clock time
- CLOCK_REALTIME

Optional wall-clock export layers MAY exist externally but remain outside synchronization scope.

---

2. Hardware & Operating Environment

2.1 Supported Hardware

Target platform:

- Raspberry Pi 4B
- Quad-core Cortex-A72
- Gigabit Ethernet
- PREEMPT_RT Linux kernel

Recommended:

- passive cooling
- fixed thermal conditions
- wired Ethernet only

2.2 GPIO Hardware Interface

GPIO 18 — Synchronization Pulse

GPIO 18 SHALL generate:

- a 10 ms HIGH pulse
- once per global second

The rising edge SHALL define the global second boundary.

The pulse SHALL remain active in all operational states except:

```
GROUND
```

Timing Constraints

Property	Requirement
Max pulse edge jitter	<20 µs
Pulse source	Virtual global clock only
Pulse generation thread	RT synchronization thread

GPIO 23 — Synchronization Health Indicator

GPIO 23 SHALL indicate synchronization health.

State	Meaning
LOW	Stable synchronized operation
HIGH	Startup, degraded sync, recovery, calibration, or fault

GPIO 23 SHALL transition LOW only after:

```
|Offset| < 100 µs
```

for a continuous:

```
10-second interval
```

2.3 External Verification

Recommended instrumentation:

- multi-channel digital oscilloscope
- logic analyzer

Minimum sample rate:

```
≥ 1 MHz
```

Measurement criteria:

- physical delta between GPIO 18 rising edges

2.4 Kernel & OS Hardening

Required Kernel Parameters

```
isolcpus=3
nohz_full=3
rcu_nocbs=3
```

Scheduling Requirements

Synchronization thread SHALL:

- execute exclusively on Core 3
- use `SCHED_FIFO`
- use priority 85

CPU Frequency Requirements

CPU governor SHALL be:

```
performance
```

Turbo and dynamic voltage scaling SHOULD be disabled.

Memory Requirements

All synchronization process memory SHALL be locked using:

```
mlockall(MCL_CURRENT | MCL_FUTURE)
```

No page faults SHALL occur in the synchronization hot path.

Time Service Isolation

The following services SHALL be disabled:

- systemd-timesyncd
- chronyd
- ntpd
- ptp4l
- phc2sys

3. Network Architecture

3.1 Network Topology

Topology assumptions:

- single-hop Ethernet
- switched Gigabit LAN
- no routing
- no NAT
- no Wi-Fi bridges
- standard MTU 1500
- no jumbo frames

3.2 IP Addressing

Static schema:

```
10.0.0.XY
```

Field	Meaning
X	Team ID (1–8)
Y	Node ID (1–3)

Example:

```
Team 4 Node 2 → 10.0.0.42
```

3.3 Discovery Mechanism

Discovery SHALL use UDP multicast.

Parameter	Value
Group	239.192.88.100
Port	47200
IP_MULTICAST_LOOP	0

`IP_MULTICAST_LOOP = 0` SHALL disable sender-local multicast loopback only.

3.4 Security Assumptions

The synchronization protocol assumes:

- trusted LAN operation
- fail-stop node behavior
- bounded congestion
- non-malicious traffic

The protocol provides NO:

- authentication
- encryption
- replay protection
- Byzantine fault tolerance
- confidentiality guarantees

Deployment outside trusted laboratory environments is NOT supported.

---

4. DRS Wire Protocol

4.1 Packet Format

All packets SHALL:

- use fixed-size binary encoding
- use network byte order (Big Endian)
- avoid dynamic allocation in the hot path
- avoid text serialization

Raw struct casting is prohibited.

All fields SHALL be serialized explicitly using:

- `htons()`
- `htonl()`
- explicit 64-bit endian conversion

4.2 Packet Size

All packets SHALL be exactly:

```
64 bytes
```

4.3 Packet Layout

Field	Type	Size	Description
Magic	uint32	4	Constant: 0x44525354 (“DRST”)
Version	uint8	1	Protocol version
MsgType	uint8	1	Message type
Flags	uint8	1	Synchronization state flags
Reserved	uint8	1	Reserved
Seq	uint16	2	Rolling sequence number
NodeID	uint32	4	Unique node ID
ElectionTerm	uint32	4	Leader election epoch
T1	uint64	8	Follower transmit timestamp
T2	uint64	8	Leader receive timestamp
T3	uint64	8	Leader transmit timestamp
T4	uint64	8	Follower receive timestamp
CRC32	uint32	4	Packet integrity
Padding	uint8[12]	12	Reserved
Total	—	64	Total packet size

4.4 Message Types

Value	Meaning
0x01	ANNOUNCE
0x02	SYNC_REQ
0x03	SYNC_RESP

4.5 Flags Field

Bit	Meaning
0	LEADER
1	HOLDOVER
2	CALIBRATED

Bit	Meaning
3	FAULT
4-7	Reserved

Reserved bits SHALL be zero.

4.6 Timestamp Semantics

Timestamp	Meaning
T1	Sync request send time
T2	Kernel-captured receive timestamp
T3	Kernel-captured transmit timestamp
T4	Sync response receive timestamp

All timestamps SHALL use:

```
CLOCK_MONOTONIC_RAW nanoseconds
```

4.7 Timestamping Mechanism

Mandatory Linux API:

```
SO_TIMESTAMPING
```

Accepted modes:

1. hardware NIC timestamping
2. kernel software timestamping

The highest precision mode supported by the platform SHALL be selected automatically.

4.8 CRC32

CRC32 SHALL use:

- IEEE 802.3 polynomial `0x04C11DB7`
- initial value `0xFFFFFFFF`
- reflected input and output

Reserved padding bytes SHALL:

- be zero-filled
- be included in CRC validation

---

5. Synchronization Mathematics & Control

5.1 Offset Estimation

```
θ = ((T2 − T1) + (T3 − T4)) / 2
```

5.2 Round Trip Delay

```
δ = (T4 − T1) − (T3 − T2)
```

5.3 Min-Delay Filter

The synchronization engine SHALL maintain:

- rolling RTT window size = 10

Only samples within:

```
minimum RTT + 10 µs
```

SHALL be accepted.

Rejected samples include:

- scheduler spikes
- interrupt storms
- retransmission artifacts
- congestion bursts

5.4 Automated Calibration

Each node SHALL perform self-latency calibration before entering synchronization states.

Calibration SHALL:

- use SO_TIMESTAMPING loopback
- use minimum 50 samples
- use minimum-delay selection
- reject outliers >20 µs from minimum

Calibration output SHALL define:

```
LatencyCorrection_ns
```

Calibration SHALL re-run:

- during startup
- after leader election
- after HOLDOVER expiration
- after synchronization fault recovery

5.5 Dual-Loop PI Clock Discipline

Step Mode

A hard phase step SHALL require:

```
3 consecutive samples confirming |θ| > 1 ms
```

Only the virtual clock MAY step.

The host OS clock SHALL NEVER be modified.

Slew Mode

If:

```
|θ| ≤ 1 ms
```

then gradual frequency correction SHALL occur through PI control.

5.6 PI Controller

```
Rate_new = Rate_old + Kp·θ + Ki·∫θdt
```

Recommended defaults:

Parameter	Value
Kp	0.05
Ki	0.005
Update Period	50 ms

Integrator SHALL be clamped to:

```
±1000 ppm equivalent
```

5.7 Slew Rate Limit

Maximum correction rate:

```
1000 ppm
```

Equivalent to:

```
50 µs per 50 ms cycle
```

---

6. Leader Election & State Machine

6.1 Election Model

The system implements a modified Bully algorithm.

Lowest NodeID SHALL win leadership.

Election timeout SHALL be randomized within:

```
250–500 ms
```

to reduce election collisions.

6.2 Node States

State	Description
GROUND	Startup stabilization
CALIBRATION	Self-latency measurement
LISTEN	Passive discovery
CANDIDATE	Election in progress
FOLLOWER	Synchronized to leader
LEADER	Authoritative clock source
HOLDOVER	Temporary freerun mode

6.3 GROUND State

After boot:

- filters SHALL be zeroed
- transmissions SHALL be suppressed
- multicast listening SHALL remain active

Duration:

```
2 seconds
```

6.4 Heartbeats

Property	Value
Leader announce interval	100 ms
Follower timeout	3 missed heartbeats
Sync exchange period	50 ms

6.5 Immediate Demotion

If a lower-ID node appears:

- current leader SHALL demote immediately
- election SHALL restart deterministically

The lower-ID node SHALL assume leadership only after:

- successful calibration
- valid heartbeat transmission

6.6 HOLDOVER Mode

If leader communication is lost:

- node SHALL enter HOLDOVER
- last stable Rate SHALL be maintained
- Offset SHALL be frozen
- GPIO 23 SHALL transition HIGH
- drift adaptation SHALL stop

Maximum HOLDOVER duration:

```
10 seconds
```

After expiration:

- node SHALL re-enter election

---

7. Real-Time Scheduling

Synchronization thread SHALL:

- use SCHED_FIFO
- use priority 85
- remain pinned to Core 3

The synchronization loop MUST periodically yield using:

```
clock_nanosleep()
```

to prevent:

- softirq starvation
- scheduler deadlock
- SSH lockout

---

8. Failure Recovery

8.1 Retry Storm Protection

Failed nodes SHALL reduce multicast rate to:

```
5 Hz
```

after:

```
5 consecutive synchronization failures
```

8.2 Filter Reset Conditions

Synchronization filters SHALL reset on:

- reboot
- sequence discontinuity
- leader change
- timestamp overflow
- phase correction >5 ms

8.3 Sequence Wraparound

Sequence numbers SHALL use unsigned 16-bit rollover semantics.

A sequence transition:

```
65535 → 0
```

SHALL be treated as valid wraparound.

Backward jumps exceeding:

```
1024 sequence values
```

SHALL invalidate synchronization state.

---

9. Lock-Free Clock Access

The virtual clock SHALL support lock-free concurrent readers.

The synchronization thread SHALL be the exclusive writer.

Reader access SHALL:

- avoid mutex blocking
- avoid dynamic allocation
- guarantee monotonic reads

Offset and Rate updates SHALL use:

- atomic 64-bit operations
- acquire/release memory ordering

---

10. Functional Requirements

ID	Requirement
F-1	Autonomous multicast discovery
F-2	Virtual global monotonic clock
F-3	Min-delay filtering
F-4	Deterministic bully election
F-5	Explicit fault recovery mapping
F-6	Lock-free telemetry
F-7	Stable convergence detection
F-8	Immediate leader demotion
F-9	Non-blocking writer precedence
F-10	Monotonic nanosecond representation
F-11	GPIO validation pulse
F-12	Fixed performance governor
F-13	SO_TIMESTAMPING support
F-14	Automatic filter reset
F-15	Fixed 64-byte packets
F-16	100 ms synchronization timeout
F-17	Step vs slew discipline

ID	Requirement
F-18	Fail-silent GPIO indicator
F-19	Seamless late joiners
F-20	Deterministic isolated startup
F-21	Static latency calibration
F-22	Retry-storm suppression
F-23	IRQ affinity isolation
F-24	Explicit endian-safe serialization
F-25	Holdover freerun support
F-26	CRC packet validation
F-27	Sequence wraparound handling
F-28	Integral windup prevention
F-29	Step confirmation hysteresis
F-30	Convergence signaling
F-31	Automated calibration
F-32	Calibration-on-election

11. Non-Functional Requirements

ID	Requirement
NF-1	Physical pulse delta <100 µs
NF-2	Pure user-space implementation
NF-3	Jitter resilience under mixed traffic
NF-4	Lock convergence within 10 s
NF-5	3-heartbeat hysteresis
NF-6	Writer never blocked by readers
NF-7	SCHED_FIFO priority 85
NF-8	mlockall memory locking
NF-9	No hot-path disk or console I/O
NF-10	Isolated Core 3 execution
NF-11	Max slew = 1000 ppm

ID	Requirement
NF-12	No dynamic heap allocation in hot path
NF-13	No packet fragmentation
NF-14	Deterministic packet parsing
NF-15	No mutex contention in sync loop
NF-16	Core 3 sanctity

---

12. systemd Deployment

Example:

```
[Unit]
Description=DRS Synchronization Service
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
ExecStart=/usr/local/bin/drs_sync
Restart=on-failure
RestartSec=5s
CPUAffinity=3
CPUSchedulingPolicy=fifo
CPUSchedulingPriority=85
LimitRTPRIO=95
LimitMEMLOCK=infinity
NoNewPrivileges=true

[Install]
WantedBy=multi-user.target
```

---

13. Performance Targets

Component	Budget
NIC timestamp jitter	<20 µs
Scheduler latency	<30 µs
Filter residual	<20 µs
GPIO generation	<20 µs

Component	Budget
Total expected worst-case	<100 µs

---

14. Architectural Constraints

The following are explicitly prohibited:

- modifying kernel clocks
- custom kernel modules
- TCP transport
- Wi-Fi synchronization operation
- distributed consensus protocols
- mutex blocking in hot path
- filesystem I/O in synchronization loop
- floating-point exceptions in RT thread

---

15. Final System Goal

The DRS cluster SHALL behave as a single deterministic distributed timing domain where all participating nodes produce externally measurable synchronization pulses aligned within:

```
<100 µs
```

under:

- PREEMPT_RT Linux
- standard user-space execution
- single-hop Ethernet
- dynamic node membership
- moderate background traffic conditions